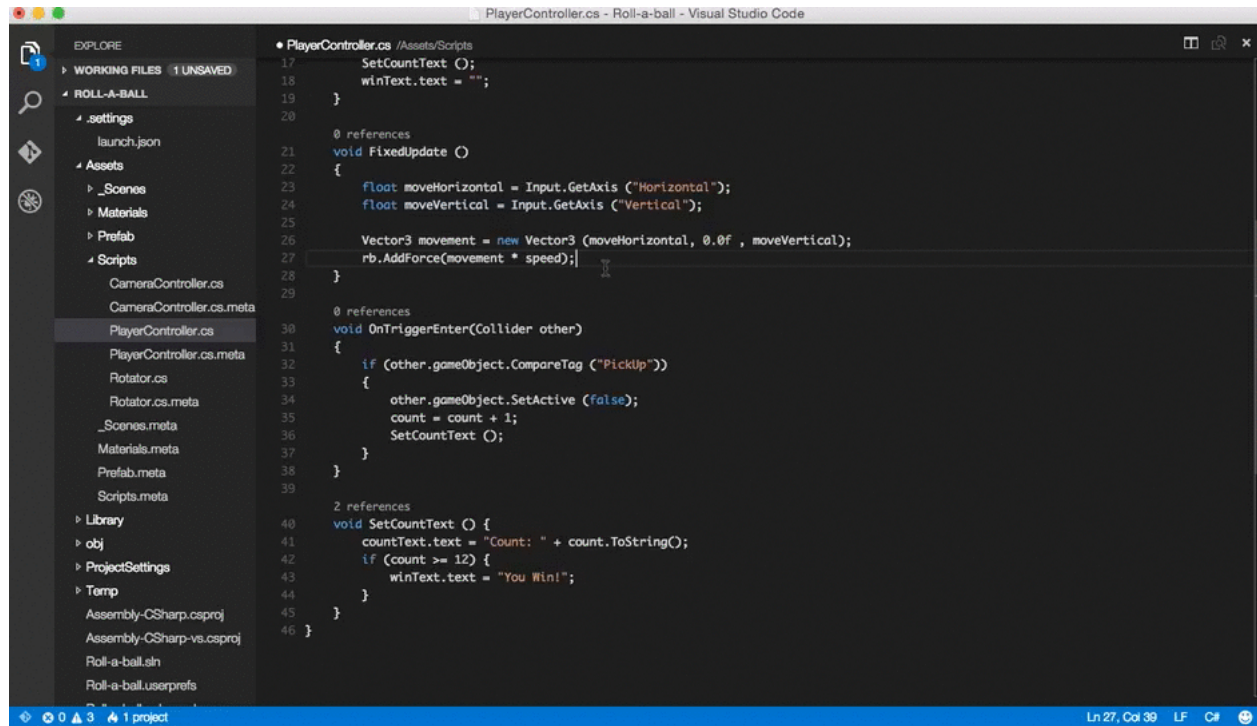


Topics Overview

 (https://vscode.dev/github/microsoft/vscode-docs/blob/main/docs/other/unity.md)

Unity Development with VS Code

Visual Studio Code can be a great companion to Unity for editing C# files. All of the C# (/docs/languages/csharp) features are supported and more. In the screen below, you can see code colorization, bracket matching, IntelliSense, CodeLens and that's just the start.



Read on to find out how to configure Unity and your project to get the best possible experience.

Prerequisites

From Using .NET in Visual Studio Code (/docs/languages/dotnet):

1. Install the .NET SDK (<https://dotnet.microsoft.com/download>), which includes the Runtime and the `dotnet` command.
2. [Windows only] Logout or restart Windows to allow changes to `%PATH%` to take effect.
3. [macOS only] To avoid seeing "Some projects have trouble loading. Please review the output for more details", make sure to install the latest stable Mono (<https://www.mono-project.com/download/>) release.

Note: This version of Mono, which is installed into your system, will not interfere with the version of MonoDevelop that is installed by Unity.

4. Install the C# extension (<https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.csharp>) from the VS Code Marketplace.
5. In the VS Code Settings editor (`⌘,`), uncheck the C# extension's **Omnisharp: Use Modern Net** setting (/docs/getstarted/settings) (`"omnisharp.useModernNet": false`).

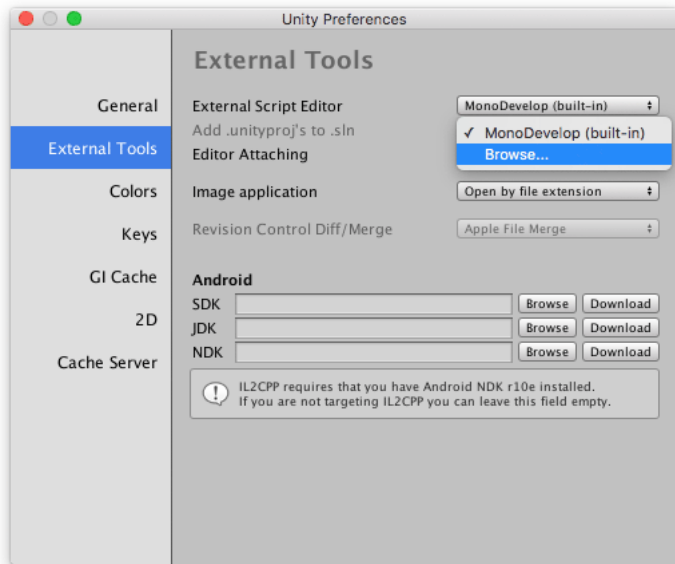
Install Build Tools for Visual Studio (Windows only)

The C# extension no longer ships with Microsoft Build Tools so they must be installed manually.

1. Download the Build Tools for Visual Studio 2022 (<https://visualstudio.microsoft.com/downloads/#build-tools-for-visual-studio-2022>).
2. Install the **.NET desktop build tools** workload. No other components are required.

Setup VS Code as Unity Script Editor

Open up **Unity Preferences**, **External Tools**, then browse for the Visual Studio Code executable as **External Script Editor**.



The Visual Studio Code executable can be found at `/Applications/Visual Studio Code.app` on macOS, `%localappdata%\Programs\Microsoft VS Code\Code.exe` on Windows by default.

Unity has built-in support for opening scripts in Visual Studio Code as an external script editor on Windows and macOS. Unity will detect when Visual Studio Code is selected as an external script editor and pass the correct arguments to it when opening scripts from Unity. Unity will also set up a default `.vscode/settings.json` with file excludes, if it does not already exist (from Unity 5.5 Release notes (<https://unity3d.com/unity/whats-new/unity-5.5.0>)).

Unity version 2019.2 or above

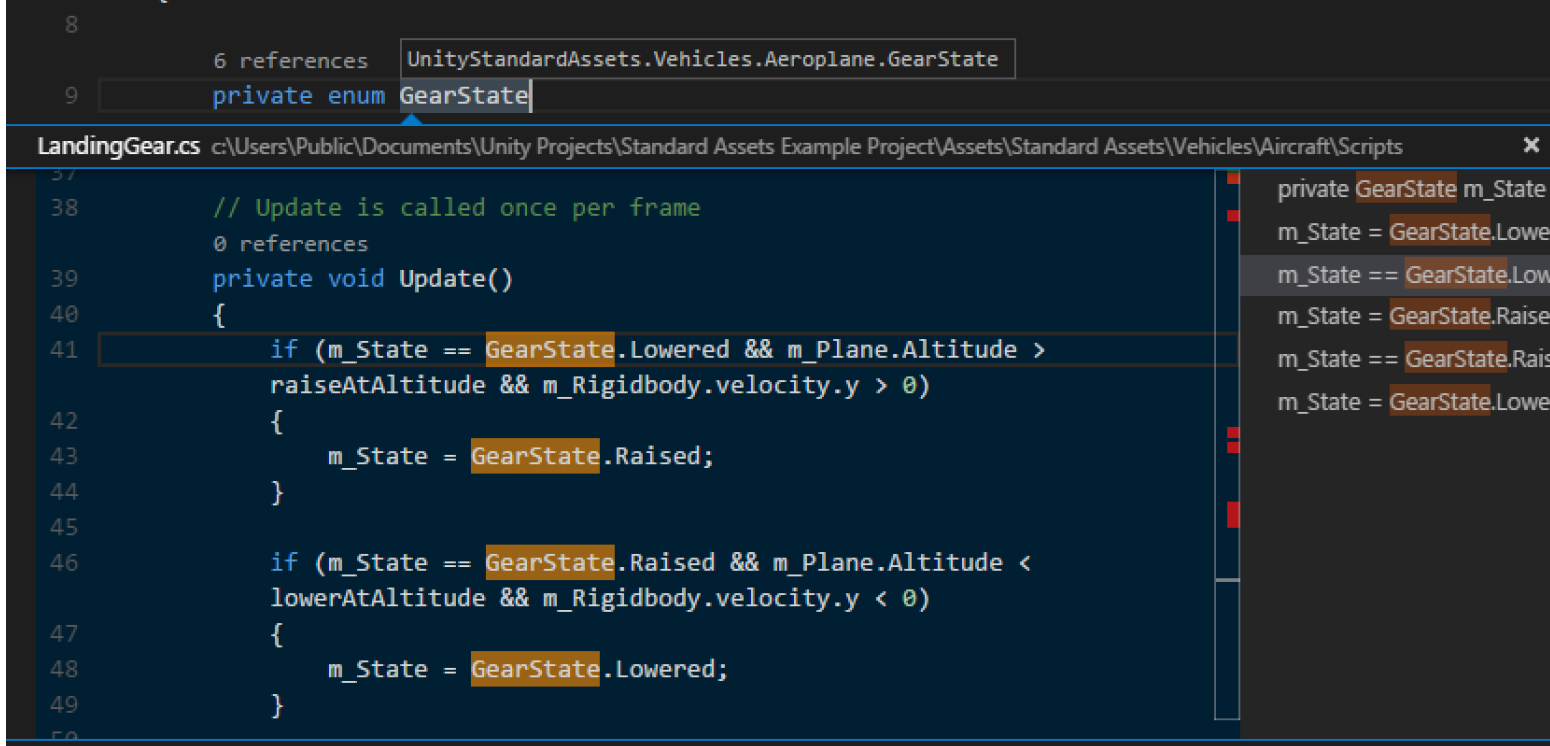
Since 2019.2 (<https://unity.com/releases/2019-2/>), it is required to use the Visual Studio Code Editor package (<https://docs.unity3d.com/Manual/com.unity.ide.vscode.html>). The built-in support for opening scripts from Unity and getting `csproj` and `sln` files generated has been removed.

Editing Evolved

With the solution file selected, you are now ready to start editing with VS Code. Here is a list of some of the things you can expect:

- Syntax Highlighting
- Bracket matching
- IntelliSense
- Snippets
- CodeLens
- Peek
- Go-to Definition
- Code Actions/Lightbulbs
- Go to symbol
- Hover

Two topics that will help you are Basic Editing (</docs/editor/codebasics>) and C# (</docs/languages/csharp>). In the image below, you can see VS Code showing hover context, peeking references and more.



Enabling code completion (For recent versions of Unity)

If you are installing VS Code for the first time, you might be missing targeting packs required for Unity's code-completion (IntelliSense) in VS Code.

Targeting pack download links:

- Windows: .NET Framework 4.7.1 Developer Pack (<https://dotnet.microsoft.com/download/dotnet-framework/net471>)
- macOS: Download .NET SDK (<https://dotnet.microsoft.com/download>)

Steps:

1. Stop VS Code or Unity running.
2. Download and install the targeting pack for your targeted framework version / preferred version from one of the above links.
3. Start Unity.
4. Create and/or open an existing script in VS Code, through Unity, and you should now see code completions.

Enabling Unity warnings

Unity has a set of custom C# warnings (<https://github.com/microsoft/Microsoft.Unity.Analyzers>), called analyzers, that check for common issues with your source code. These analyzers ship out of the box with Visual Studio but need to be set up manually in Visual Studio Code.

Due to how Unity handles its .csproj files, it does not seem possible to install packages automatically. You will need to download the analyzers from the NuGet website (<https://www.nuget.org/packages/Microsoft.Unity.Analyzers/>) manually. Make sure to download the .nupkg file, the source code from GitHub will not work.

When you're done, open the package file using a tool such as 7zip and extract Microsoft.Unity.Analyzers.dll onto your project's root folder. You can place it inside a folder named NuGet, for example. Do not place it inside Assets or Packages, as that will cause Unity to try to process the .dll, which will make it output an error in the console.

Note: 7zip cannot open a .nupkg file by right-click and **Open with**. Instead, you have to open the 7zip application, navigate to the file, and then select **Extract**.

Next, create an omnisharp.json file at the root folder of your project, as explained here (<https://www.strathweb.com/2019/04/roslyn-analyzers-in-code-fixes-in-omnisharp-and-vs-code/>). Analyzer support in OmniSharp is experimental at the moment, so we need to enable it explicitly. We also need to point it to the .dll file we just extracted.

Your omnisharp.json file should end up looking like this:

```
{
  "RoslynExtensionsOptions": {
    "EnableAnalyzersSupport": true,
    "LocationPaths": [".\\NuGet\\microsoft.unify.analyzers.1.14.0\\analyzers\\dotnet\\cs"]
  }
}
```

where ".\\NuGet\\microsoft.unify.analyzers.1.14.0\\analyzers\\dotnet\\cs" is a relative path pointing to the folder containing the .dll file. Depending on where you placed it, your path may look different.

The Unity analyzers should now be working in your project. You can test them by creating an empty FixedUpdate() method inside one of your MonoBehaviour classes, which should trigger a The Unity message 'FixedUpdate' is empty warning (UNT0001 (<https://github.com/microsoft/Microsoft.Unity.Analyzers/blob/main/doc/UNT0001.md>)).

Note that while it is possible to activate these analyzers, the suppressors they ship with the package (that turn off other C# warnings that may conflict with these custom ones) may not be picked up by OmniSharp at the moment, according to this thread (<https://github.com/microsoft/Microsoft.Unity.Analyzers/issues/122#issuecomment-743747554>). You can still turn off specific rules manually by following these steps:

1. Create a .editorconfig file in your project's root folder (next to Unity's .csproj files).
2. Add the following contents to the file:

```
root=true

[*.cs]
dotnet_diagnostic.IDE0051.severity = none
```

`root=true` tells OmniSharp that this is your project root and it should stop looking for parent `.editorconfig` files outside of this folder.

`dotnet_diagnostic.IDE0051.severity = none` is an example of turning off the analyzer with ID `IDE0051` by setting its severity level to `none`. You can read more about these settings in the Analyzer overview (<https://learn.microsoft.com/visualstudio/code-quality/use-roslyn-analyzers>). You can add as many of these rules as you want to this file.

`[*.cs]` indicates that our custom rules should apply to all C# scripts (files with the `.cs` extension).

You are now ready to code in Visual Studio Code, while getting the same warnings as you would when using Visual Studio!

Next steps

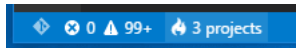
Read on to learn more about:

- Basic Editing (</docs/editor/codebasics>) - Learn about the powerful VS Code editor.
- Code Navigation (</docs/editor/editingevolved>) - Move quickly through your source code.
- C# (</docs/languages/csharp>) - learn about the C# support in VS Code

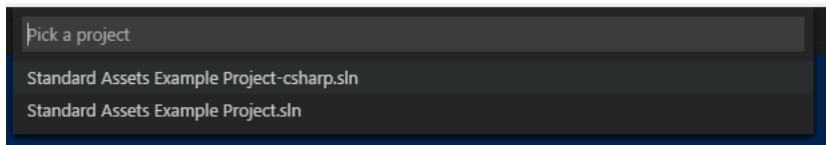
Common questions

I don't have IntelliSense

You need to ensure that your solution is open in VS Code (not just a single file). Open the folder with your solution and you usually will not need to do anything else. If for some reason VS Code has not selected the right solution context, you can change the selected project by clicking on the OmniSharp flame icon on the status bar.



Choose the `-CSharp` version of the solution file and VS Code will light up.



How can I change the file exclusions?

Unity creates a number of additional files that can clutter your workspace in VS Code. You can easily hide these so that you can focus on the files you actually want to edit.

To do this, add the following JSON to your workspace settings (</docs/getstarted/settings>). By adding these excludes to your workspace settings, you will not change your global user settings and it allows anyone also working on the project to have the same file excludes.

```
// Configure glob patterns for excluding files and folders.
"files.exclude": {
  "**/.git": true,
  "**/.DS_Store": true,
  "**/*.meta": true,
  "**/*.meta.meta": true,
  "**/*.unity": true,
  "**/*.unityproj": true,
  "**/*.mat": true,
  "**/*.fbx": true,
  "**/*.FBX": true,
  "**/*.tga": true,
  "**/*.cubemap": true,
  "**/*.prefab": true,
  "**/Library": true,
  "**/ProjectSettings": true,
  "**/Temp": true
}
```

As you can see below this will clean things up a lot...

Before	After
Scripts CameraSwitch.cs CameraSwitch.cs.meta LevelReset.cs LevelReset.cs.meta ParticleSceneControls.cs ParticleSceneControls.cs.meta PlaceTargetWithMouse.cs PlaceTargetWithMouse.cs.meta SlowMoButton.cs SlowMoButton.cs.meta	Scripts CameraSwitch.cs LevelReset.cs ParticleSceneControls.cs PlaceTargetWithMouse.cs SlowMoButton.cs

To edit this directly within VS Code Settings editor, go to **File > Preferences > Settings (Code > Preferences > Settings** on macOS). Switch to the **Workspace** tab and then type "files exclude" into the Settings editor search bar. Add a glob pattern similar to the pattern shown below by clicking the **Add Pattern** button for the **Files: Exclude** setting. You will need to add each pattern separately.

```

**/.git
**/*.fbx
**/*.FBX
**/*.mat
**/*.meta
**/*.meta
**/*.prefab
**/*.unityproj
**/Library
**/Packages
**/ProjectSettings
**/Temp

```

Was this documentation helpful?

Yes

No

11/4/2022

IN THIS ARTICLE

- Prerequisites
 - Setup VS Code as Unity Script Editor
 - Unity version 2019.2 or above
 - Editing Evolved
 - Enabling code completion (For recent versions of Unity)
 - Enabling Unity warnings
- Next steps
- Common questions

 Tweet this link

([https://twitter.com/intent/tweet?](https://twitter.com/intent/tweet?original_referer=https://code.visualstudio.com/docs/other/unity&ref_src=twsrc%5Etfw&text=Visual%20Studio%20Code%20and%20Unity&tw_p=tweetbutton&url=https://code.visualstudio.com/docs/other/unity)
[original_referer=https://code.visualstudio.com/docs/other/unity&ref_src=twsrc%5Etfw&text=Visual%20Studio%20Code%20and%20Unity&tw_p=tweetbutton&url=https://code.visualstudio.com/docs/other/unity](https://code.visualstudio.com/docs/other/unity&ref_src=twsrc%5Etfw&text=Visual%20Studio%20Code%20and%20Unity&tw_p=tweetbutton&url=https://code.visualstudio.com/docs/other/unity))

 Subscribe(/feed.xml)

 Ask questions(<https://stackoverflow.com/questions/tagged/vscode>)

 Follow @code(<https://go.microsoft.com/fwlink/?LinkID=533687>)

 Request features(<https://go.microsoft.com/fwlink/?LinkID=533482>)

 Report issues(<https://www.github.com/Microsoft/vscode/issues>)

 Watch videos(https://www.youtube.com/channel/UCs5Y5_7XK8HLDX0SLNwkd3w)

Hello from Seattle. Follow @code (<https://go.microsoft.com/fwlink/?LinkID=533687>)

Star 144,034

Support (<https://support.serviceshub.microsoft.com/supportforbusiness/create?sapId=d66407ed-3967-b000-4cfb-2c318cad363d>) Privacy (<https://privacy.microsoft.com/privacystatement>)
Terms of Use (<https://www.microsoft.com/legal/terms-of-use>) License (/License)